

INSTITUTO INFNET

NEUZA GIOVANNA DE OLIVEIRA

DISCIPLINA: Projeto de Bloco: Desenvolvimento Back-End

Modelagem de Classes e Relacionamentos

MediCare — Plataforma de Agendamento e Gerenciamento de Consultas Médicas

---

1. Introdução

Na primeira etapa foram definidos os requisitos do MediCare e, na segunda, esses requisitos foram formalizados em treze casos de uso envolvendo três atores: Paciente, Médico e Administrador. Esta terceira etapa corresponde ao início da fase de Elaboração do RUP e tem como objetivo transformar os casos de uso em um modelo estrutural do sistema.

Para isso, este documento apresenta: a lista de classes candidatas extraídas dos casos de uso e a decisão tomada para cada uma; a matriz de associação entre as classes selecionadas; o dicionário de dados com classes, atributos e associações; o diagrama de classes UML; e a primeira parte da implementação, um software em C# que lê e grava os dados do sistema em arquivos CSV.

A tecnologia escolhida para o projeto é **C#/ASP.NET**. Nas próximas etapas a solução será desenvolvida com Razor Pages, reutilizando as classes e a camada de dados implementadas aqui.

---

2. Identificação das Classes Candidatas

As classes candidatas foram levantadas a partir dos substantivos que aparecem na descrição do sistema (TP1) e nos casos de uso (TP2). Cada termo foi avaliado: alguns representam informações que precisam ser guardadas pelo sistema e viraram classes; outros são apenas atributos de uma classe, operações, ou formas de visualizar dados que já existem em outras classes.

Termo identificado	Origem	Decisão	Justificativa
Usuário	UC01, UC02, UC12	Classe abstrata **Usuario**	Paciente, Médico e Administrador possuem dados de acesso em comum (nome, e-mail, senha, situação). Esses dados ficam em uma superclasse.

Termo identificado	Origem	Decisão	Justificativa
Paciente	TP1, UC01, UC03–UC08, UC11	Classe **Paciente**	Ator principal; agenda e gerencia consultas.
Médico	TP1, UC01, UC05, UC08–UC11	Classe **Medico**	Ator que define disponibilidade e atende consultas.
Administrador	TP1, UC02, UC12, UC13	Classe **Administrador**	Ator responsável pelo gerenciamento da plataforma.
Especialidade	TP1 (pesquisa por especialidade), UC03	Classe **Especialidade**	É o critério de pesquisa de médicos. Como classe, evita nomes digitados de formas diferentes.
Horário disponível / disponibilidade	UC04, UC05, UC09	Classe **HorarioDisponivel**	Cada horário cadastrado pelo médico precisa ser guardado com data, hora e situação (livre ou reservado).
Modalidade de atendimento	TP1, UC05	Enumeração **Modalidade**	Possui apenas dois valores fixos: presencial e teleconsulta. É definida pelo médico em cada horário.
Local do atendimento	TP1 (visualizar local da consulta presencial)	Classe **LocalAtendimento**	Endereço onde o médico atende presencialmente; é exibido ao paciente.
Consulta / agendamento	UC05, UC06, UC07, UC10, UC13	Classe **Consulta**	Registro central do sistema: liga um paciente a um horário de um médico.
Situação da consulta	UC06, UC07	Enumeração **StatusConsulta**	A consulta pode estar agendada, cancelada ou realizada.
Conta, credenciais, perfil	UC01, UC02, UC08	Atributos de Usuario e subclasses	Não têm identidade própria; são os dados do próprio usuário (e-mail, senha, telefone etc.).
Agenda do médico	UC10	Descartada como classe	É uma consulta sobre os horários e consultas de um médico; não há dado novo a guardar.
Teleconsulta	UC11	Representada por Consulta + Modalidade	É uma consulta cujo horário tem modalidade Teleconsulta. Não há, até esta etapa, informação própria que justifique uma classe separada.
Cancelamento, reagendamento	UC06, UC07	Operações de Consulta	São ações sobre a consulta, que alteram o status ou o horário.
Critérios de pesquisa	UC03	Parâmetro de operação	Usados apenas no momento da busca, não são armazenados.
Sistema / plataforma	TP1	Descartada	Representa o próprio sistema, não uma entidade do domínio.

Resultado: **oito classes** (Usuario, Paciente, Medico, Administrador, Especialidade, LocalAtendimento, HorarioDisponivel e Consulta) e **duas enumerações** (Modalidade e StatusConsulta).

---

3. Matriz de Associação

A matriz mostra como as classes se relacionam. Cada célula deve ser lida da linha para a coluna e indica a multiplicidade da classe da coluna em relação a um objeto da classe da linha. Por exemplo, na linha *Paciente* e coluna *Consulta*, “0..*” significa que um paciente pode ter nenhuma ou várias consultas. “Herança” indica generalização, e “—” indica que não há associação direta.

	Usuario	Paciente	Medico	Administr.	Especialid.	Local Atend.	Horário Disp.	Consulta
Usuario	—	herança	herança	herança	—	—	—	—
Paciente	herança	—	—	—	—	—	—	0..*
Medico	herança	—	—	—	1	0..*	0..*	—
Administrador	herança	—	—	—	—	—	—	—
Especialidade	—	—	0..*	—	—	—	—	—
LocalAtendimento	—	—	1	—	—	—	0..*	—
HorarioDisponivel	—	—	1	—	—	0..1	—	0..*
Consulta	—	1	—	—	—	—	1	—

O Administrador não possui associação estrutural com as demais classes. Nos casos de uso UC12 e UC13 ele consulta e altera usuários e consultas, mas isso é feito por operações do sistema e não gera um vínculo que precise ser armazenado. Por isso a linha dele aparece vazia, exceto pela herança.

---

4. Dicionário de Dados

4.1 Classes

Classe	Descrição
Usuario (abstrata)	Dados comuns a todos que acessam a plataforma: identificação, credenciais e situação da conta.
Paciente	Usuário que pesquisa médicos e agenda, cancela e reagenda consultas.
Medico	Usuário profissional de saúde que cadastra horários e atende consultas.
Administrador	Usuário que gerencia os demais usuários e acompanha os agendamentos.
Especialidade	Área de atuação médica usada na pesquisa de profissionais.
LocalAtendimento	Endereço onde um médico realiza atendimentos presenciais.
HorarioDisponivel	Intervalo de tempo cadastrado pelo médico, com modalidade definida, que pode ser reservado por uma consulta.
Consulta	Agendamento de um paciente em um horário disponível de um médico.

4.2 Atributos

Os atributos marcados como *referência* não aparecem no diagrama, porque lá estão representados pelas linhas de associação. Eles existem na implementação, onde a ligação entre os objetos é feita pelo identificador (Id) gravado nos arquivos CSV.

Usuario

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único dentro do tipo de usuário.
nome	string	Sim	Nome completo.
email	string	Sim	E-mail usado no login; não pode se repetir.
senhaHash	string	Sim	Hash SHA-256 da senha. A senha em texto não é armazenada.
ativo	bool	Sim	Indica se a conta está ativa. O administrador pode desativá-la.
dataCadastro	DateTime	Sim	Data e hora de criação da conta.

Paciente (herda de Usuario)

Atributo	Tipo	Obrig.	Descrição
telefone	string	Sim	Telefone de contato.
dataNascimento	Date	Sim	Data de nascimento.

Medico (herda de Usuario)

Atributo	Tipo	Obrig.	Descrição
crm	string	Sim	Registro no Conselho Regional de Medicina, com a UF.
telefone	string	Sim	Telefone profissional.
especialidadeId	int (referência)	Sim	Especialidade do médico.

Administrador (herda de Usuario): não possui atributos próprios.

Especialidade

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
nome	string	Sim	Nome da especialidade (ex.: Cardiologia).

LocalAtendimento

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
medicoId	int (referência)	Sim	Médico que atende neste local.
nome	string	Sim	Nome da clínica ou consultório.
endereco	string	Sim	Logradouro, número e complemento.
cidade	string	Sim	Cidade do atendimento.

HorarioDisponivel

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
medicoId	int (referência)	Sim	Médico dono do horário.
data	Date	Sim	Dia do atendimento.
horaInicio	Time	Sim	Hora de início.
horaFim	Time	Sim	Hora de término.
modalidade	Modalidade	Sim	Presencial ou Teleconsulta.
localId	int (referência)	Não	Local do atendimento. Preenchido apenas quando a modalidade é Presencial.
disponivel	bool	Sim	Verdadeiro enquanto o horário não estiver reservado por uma consulta.

Consulta

Atributo	Tipo	Obrig.	Descrição
id	int	Sim	Identificador único.
pacienteId	int (referência)	Sim	Paciente que agendou.
horarioId	int (referência)	Sim	Horário reservado. Muda quando a consulta é reagendada.
status	StatusConsulta	Sim	Agendada, Cancelada ou Realizada.
dataAgendamento	DateTime	Sim	Momento em que a consulta foi agendada.

4.3 Enumerações

Enumeração	Valores	Descrição
Modalidade	Presencial, Teleconsulta	Forma de atendimento definida pelo médico para cada horário.
StatusConsulta	Agendada, Cancelada, Realizada	Situação da consulta. Somente consultas agendadas podem ser canceladas, reagendadas ou concluídas.

4.4 Associações

Associação	Classes	Multiplicidade	Descrição
herança	Paciente, Medico, Administrador → Usuario	—	Os três tipos de usuário herdam os dados de acesso de Usuario.
possui	Medico — Especialidade	0..* : 1	Cada médico tem uma especialidade; uma especialidade pode ter vários médicos (ou nenhum).
disponibiliza (composição)	Medico — HorarioDisponivel	1 : 0..*	O médico cadastra seus horários. Um horário só existe vinculado ao seu médico.
atende em (composição)	Medico — LocalAtendimento	1 : 0..*	O médico cadastra os locais onde atende presencialmente.
ocorre em	HorarioDisponivel — LocalAtendimento	0..* : 0..1	Horário presencial aponta para um local; horário de teleconsulta não tem local.
agenda	Paciente — Consulta	1 : 0..*	Um paciente pode ter várias consultas; cada consulta é de um paciente.
ocupa	Consulta — HorarioDisponivel	0..* : 1	Cada consulta ocupa um horário. Um mesmo horário pode aparecer em mais de uma consulta ao longo do tempo (por exemplo, após um cancelamento), mas apenas uma pode estar com status Agendada.

---

5. Diagrama de Classes

O diagrama abaixo foi elaborado em notação UML a partir das classes e associações descritas nas seções anteriores. Ele mostra os atributos relevantes, as principais operações e a multiplicidade de cada associação.

O diagrama está na página seguinte, em orientação paisagem, para facilitar a leitura.

Algumas decisões do modelo:

- A generalização em **Usuario** permite que o login (UC02) e a ativação/desativação de contas (UC12) funcionem da mesma forma para os três atores.
- A **modalidade** fica no horário, e não na consulta, porque na TP1 é o médico quem define se um horário é presencial ou teleconsulta. A consulta herda essa informação do horário que ocupa.
- As operações **reservar()** e **liberar()** de HorarioDisponivel implementam o passo “o sistema atualiza a disponibilidade do médico” (UC05) e “o horário pode voltar a ser disponibilizado” (UC06).
- As operações **cancelar()** e **reagendar()** de Consulta correspondem aos casos de uso UC06 e UC07. **concluir()** marca a consulta como realizada.

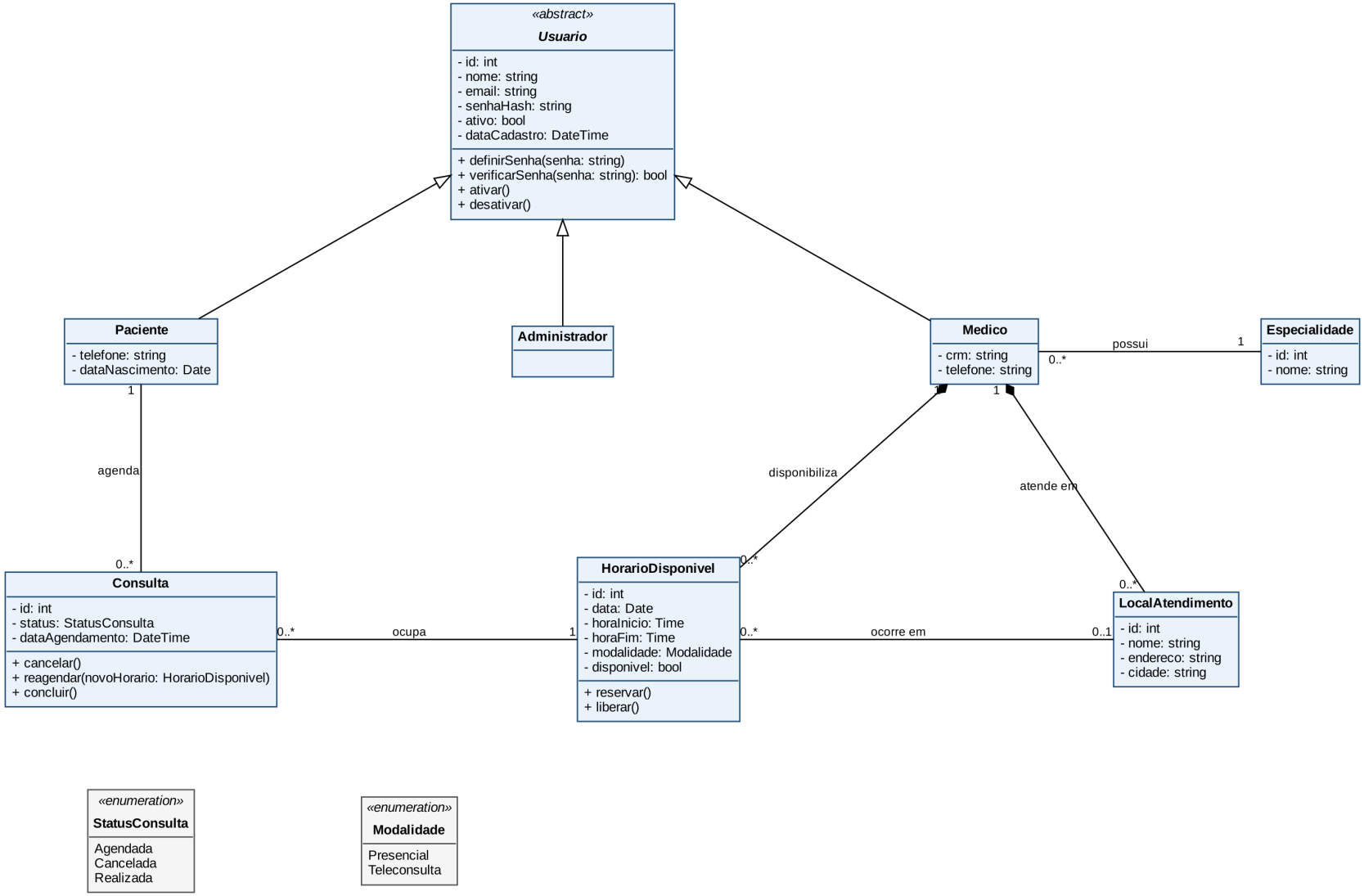


Figura 1 — Diagrama de Classes do sistema MediCare. Fonte: elaboração própria (Graphviz), com auxílio de ferramenta de inteligência artificial.

6. Implementação: Leitura e Escrita de Arquivos CSV

6.1 Organização da solução

A solução foi criada em C# com .NET 8 e está dividida em dois projetos. O **MediCare.Core** é uma biblioteca com as classes do diagrama, a leitura e escrita de CSV e as regras de agendamento; ele será referenciado pela aplicação Razor Pages nas próximas entregas. O **MediCare.CsvApp** é um programa de console usado nesta etapa para testar a leitura e a gravação dos dados.

```
MediCare/
├── MediCare.sln
├── dados/                          arquivos CSV de exemplo
├── src/MediCare.Core/
│   ├── Models/                     Usuario, Paciente, Medico, Administrador, Especialidade,
│   │                               LocalAtendimento, HorarioDisponivel, Consulta, Enums
│   ├── Csv/CsvParser.cs            leitura e escrita no formato CSV
│   ├── Csv/CsvValor.cs             conversão de datas, horas, números e enums
│   ├── Repositories/               um repositório por arquivo CSV
│   └── Services/AgendamentoService.cs   regras dos casos de uso UC03 a UC07
└── src/MediCare.CsvApp/Program.cs  aplicação de console
```

6.2 Arquivos CSV

Cada classe concreta do modelo é gravada em um arquivo próprio. A primeira linha é sempre o cabeçalho, que é conferido na leitura: se o arquivo estiver com colunas diferentes do esperado, o programa informa o erro em vez de carregar dados errados. Datas e horas são gravadas no formato ISO (2026-12-01, 09:30) para que o arquivo não dependa da configuração regional do computador.

Arquivo	Classe	Colunas
especialidades.csv	Especialidade	Id, Nome
pacientes.csv	Paciente	Id, Nome, Email, SenhaHash, Telefone, DataNascimento, Ativo, DataCadastro
medicos.csv	Medico	Id, Nome, Email, SenhaHash, Crm, Telefone, EspecialidadeId, Ativo, DataCadastro
administradores.csv	Administrador	Id, Nome, Email, SenhaHash, Ativo, DataCadastro
locais_atendimento.csv	LocalAtendimento	Id, MedicoId, Nome, Endereco, Cidade
horarios.csv	HorarioDisponivel	Id, MedicoId, Data, HoraInicio, HoraFim, Modalidade, LocalId, Disponivel
consultas.csv	Consulta	Id, PacienteId, HorarioId, Status, DataAgendamento

Exemplo do arquivo *locais_atendimento.csv*. Como os endereços contêm vírgulas, esses campos são gravados entre aspas; aspas dentro do texto são duplicadas, conforme o padrão RFC 4180:

```
Id,MedicoId,Nome,Endereco,Cidade
1,1,Clínica Coração,"Rua das Laranjeiras, 120, sala 305",Rio de Janeiro
2,2,Consultório Pele & Saúde,"Av. Rio Branco, 45, 8º andar",Rio de Janeiro
3,3,"Espaço Infantil ""Crescer""","Rua Augusta, 900",São Paulo
```

Exemplo do arquivo *horarios.csv*. Na teleconsulta, a coluna LocalId fica vazia:

```
Id,MedicoId,Data,HoraInicio,HoraFim,Modalidade,LocalId,Disponivel
1,1,2026-12-01,09:00,09:30,Presencial,1,false
2,1,2026-12-01,09:30,10:00,Presencial,1,true
3,1,2026-12-02,14:00,14:30,Teleconsulta,,true
4,2,2026-12-03,10:00,10:30,Presencial,2,true
5,2,2026-12-03,16:00,16:30,Teleconsulta,,true
6,3,2026-12-04,08:00,08:30,Presencial,3,true
```

6.3 Leitura e escrita do formato CSV

A classe **CsvParser** lê o arquivo caractere por caractere, controlando se está dentro de um campo entre aspas. Assim, vírgulas e quebras de linha dentro de aspas fazem parte do valor e não separam campos. Na escrita, o método *Escapar* só coloca aspas quando o valor precisa delas:

```
public static string FormatarLinha(IEnumerable<string> campos) =>
    string.Join(Separador, campos.Select(Escapar));

private static string Escapar(string valor)
{
    if (valor.IndexOfAny(new[] { Separador, '"', '\n', '\r' }) < 0)
        return valor;
    return $"\"{valor.Replace("\"", "\"\"")}\"";
}
```

6.4 Repositório genérico

A classe abstrata **CsvRepository<T>** concentra o que é comum a todos os arquivos: carregar os registros, validar o cabeçalho, gerar o próximo Id e gravar. A gravação é feita primeiro em um arquivo temporário, que depois substitui o original, para que uma falha no meio da escrita não deixe o CSV incompleto.

```
public void Salvar()
{
    Directory.CreateDirectory(Path.GetDirectoryName(Path.GetFullPath(_caminho))!);

    var conteudo = new StringBuilder();
    conteudo.AppendLine(CsvParser.FormatarLinha(Cabecalho));
    foreach (var item in _itens.OrderBy(i => i.Id))
        conteudo.AppendLine(CsvParser.FormatarLinha(ParaCampos(item)));

    // Grava primeiro em um arquivo temporário para não corromper o original em caso de falha.
    var temporario = _caminho + ".tmp";
    File.WriteAllText(temporario, conteudo.ToString(), new UTF8Encoding(false));
    File.Move(temporario, _caminho, overwrite: true);
}
```

Cada repositório concreto informa apenas o cabeçalho e como converter uma linha em objeto e um objeto em linha. Exemplo do repositório de consultas:

```
public class ConsultaRepository(string caminho) : CsvRepository<Consulta>(caminho)
{
    protected override string[] Cabecalho => ["Id", "PacienteId", "HorarioId", "Status", "DataAgendamento"];

    protected override Consulta DeCampos(IReadOnlyList<string> c) => new(
        id: CsvValor.ParaInt(c[0]),
        pacienteId: CsvValor.ParaInt(c[1]),
        horarioId: CsvValor.ParaInt(c[2]),
        status: CsvValor.ParaEnum<StatusConsulta>(c[3]),
        dataAgendamento: CsvValor.ParaDataHora(c[4]));

    protected override IEnumerable<string> ParaCampos(Consulta c) =>
    [
        CsvValor.De(c.Id), CsvValor.De(c.PacienteId), CsvValor.De(c.HorarioId),
        CsvValor.De(c.Status), CsvValor.De(c.DataAgendamento)
    ];
}
```

6.5 Regras de agendamento

A classe **AgendamentoService** segue os fluxos descritos na TP2. No agendamento (UC05), o sistema verifica o paciente, reserva o horário, registra a consulta e grava os arquivos. Se o horário já estiver reservado, *Reservar()* lança uma exceção e nada é gravado.

```
public Consulta Agendar(int pacienteId, int horarioId)
{
    var paciente = dados.Pacientes.ObterPorId(pacienteId)
        ?? throw new InvalidOperationException("Paciente não encontrado.");
    if (!paciente.Ativo)
        throw new InvalidOperationException("Paciente desativado não pode agendar consultas.");

    var horario = ObterHorario(horarioId);
    horario.Reservar();

    var consulta = dados.Consultas.Adicionar(new Consulta(pacienteId, horarioId));
    Salvar();
    return consulta;
}
```

No cancelamento (UC06), a consulta passa para Cancelada e o horário é liberado. No reagendamento (UC07), o novo horário precisa ser do mesmo médico e estar livre; a consulta passa a apontar para ele e o horário anterior é liberado.

6.6 Execução

O programa recebe como argumento a pasta dos arquivos CSV:

```
dotnet run --project src/MediCare.CsvApp -- dados
```

Trecho de uma execução real, em que o paciente 2 pesquisa médicos, agenda uma consulta com a Dra. Carla Mendes, reagenda para uma teleconsulta e, em seguida, a consulta 1 é cancelada (as entradas digitadas estão após os dois-pontos):

```
MediCare — dados carregados de '/tmp/demo/MediCare/dados'

Opção: 1
  [1] Clínica Geral
  [2] Cardiologia
  [3] Dermatologia
  [4] Pediatria
Id da especialidade (Enter para todas):
  [1] Carla Mendes — Cardiologia — CRM-RJ 123456
  [2] Daniel Rocha — Dermatologia — CRM-RJ 654321
  [3] Elisa Martins — Pediatria — CRM-SP 112233

Opção: 4
Id do paciente: 2
Id do médico: 1
  [2] 01/12/2026 09:30-10:00 — Presencial — Clínica Coração, Rua das Laranjeiras, 120, sala 305, Rio de Janeiro
  [3] 02/12/2026 14:00-14:30 — Teleconsulta
Id do horário: 2
Consulta 2 agendada.

Opção: 6
Id da consulta: 2
  [3] 02/12/2026 14:00-14:30 — Teleconsulta
Id do novo horário: 3
Consulta reagendada.

Opção: 5
Id da consulta: 1
Consulta cancelada e horário liberado.
```

Conteúdo de *consultas.csv* após a execução:

```
Id,PacienteId,HorarioId,Status,DataAgendamento
1,1,1,Cancelada,2026-11-05T18:20:00
2,2,3,Agendada,2026-11-09T09:22:11
```

Também foram testadas as situações de erro: agendar um horário já reservado, alterar uma consulta cancelada e abrir um CSV com data inválida. Nos dois primeiros casos o programa exibe a mensagem e não altera os arquivos; no terceiro, informa o arquivo e o registro com problema. Além disso, ao carregar e gravar novamente os sete arquivos de exemplo, o conteúdo gerado foi idêntico ao original.

---

7. Conclusão

Nesta etapa, os casos de uso do MediCare foram analisados para identificar as classes do sistema. Dos termos levantados, oito se tornaram classes e dois se tornaram enumerações; os demais foram tratados como atributos, operações ou descartados, com a justificativa registrada. A matriz de associação, o dicionário de dados e o diagrama de classes descrevem a estrutura que dará suporte às funcionalidades definidas nas etapas anteriores.

A implementação iniciada em C# já lê e grava todas as entidades do modelo em arquivos CSV e executa as regras de agendamento, cancelamento e reagendamento. A biblioteca MediCare.Core foi separada da aplicação de console justamente para ser reaproveitada na interface Razor Pages, que será desenvolvida nas próximas entregas.

Nota sobre o uso de inteligência artificial

Na elaboração deste trabalho foi utilizada uma ferramenta de inteligência artificial (Claude, da Anthropic) como apoio na organização do documento, na geração do diagrama de classes e na escrita do código. Todo o conteúdo foi construído a partir dos requisitos e casos de uso definidos na TP1 e na TP2, e foi revisado e testado pela autora.